PROGRAM

```
# Import necessary libraries import numpy as np import pandas
as pd
import scipy.stats as stats import
statsmodels.api as sm import
matplotlib.pyplot as plt
```

1. NumPy: Numerical Python

```
# Create a NumPy array and perform basic operations

arr1 = np.array([12, 18, 25, 30, 45])
arr2 = np.array([3, 5, 7, 9, 11])
print("Array 1:", arr1)
print("Array 2:", arr2) #Addition
print("Addition:", arr1 + arr2) #
Mean
print("Mean of Array 1:", np.mean(arr1)) #
Standard Deviation
print("Standard Deviation:", np.std(arr1))
```

2. Pandas: Data manipulation and analysis

```
# Create a Pandas DataFrame
student_data = {"Name": ["Asha", "Rahul", "Divya", "Kiran", "Manoj"], "Marks": [78, 92,
85, 92, 74]}
df = pd.DataFrame(student_data) print(df)
print("\nMean Marks:", df["Marks"].mean())
print("Median Marks:", df["Marks"].median())
  print("Mode Marks:")


 df["Marks"].mode())
```

#3. SciPy model

```
from scipy.stats import ttest_ind # Sample datasets class_A =
[72, 75, 78, 74, 77]
class_B = [68, 70, 69, 71, 72]

# Perform t-test

t_value, p_value = ttest_ind(class_A, class_B) print("T-
Statistic:", t_value)
print("P-Value:", p_value) if p_value < 0.05:

print("There is a significant difference between the two groups.") else:
  print("No significant difference between the two groups.")
```

4. Statsmodels: Statistical models # Fit a

```
simple linear regression model import
statsmodels.api as sm temperature = [25, 28,
30, 32, 35]

sales = [100, 130, 170, 210, 260]
X = sm.add_constant(temperature) model
= sm.OLS(sales, X).fit()
print("Statsmodels OLS Regression Results")
print(model.summary())
new_temp = sm.add_constant([38], has_constant='add') prediction =
model.predict(new_temp)
```

```
print("Predicted Ice Cream Sales:", prediction[0])
```

5. Matplotlib: Plotting graphs

```
# Plot a graph
temperature = [25, 28, 30, 32, 35, 38, 40]
sales = [100, 130, 170, 210, 260, 300, 340]
# Scatter Plot

plt.scatter(temperature, sales, color="purple", s=120, marker="D")

# Labels and Title plt.title("Temperature vs Ice Cream Sales") plt.xlabel("Temperature (°C)")

plt.ylabel("Ice Cream Sales") # Add Grid
plt.grid(linestyle="--", alpha=0.7)

# Show Plot
plt.show()

import numpy as np
```

#Create an array from 0 to 9

```
array_arange = np.arange(20)
print("Array using arange():", array_arange) # Create
a 3x3 array with random values array_random =
np.random.rand(4,4 )

print("Array using random.rand():\n", array_random)
```

#Create a 2x2 empty array

```
array_empty = np.empty((3,3))
print("Array using empty():\n", array_empty)
```

3.b

#Create a 3x3 array filled with zeros

```
array_zeros = np.zeros((5,5))
print("Array filled with zeros:\n", array_zeros) # Create
a 3x3 array filled with ones array_ones =
np.ones((4, 4))
print("Array filled with ones:\n", array_ones)
```

3.c

```
#Create a structured array for the student database

student_dtype = np.dtype([('ID','i4'),('Name','U10'),('Age','i4'),('Marks','f4'),('Grade', 'U2')])
students = np.array([ (1,'Alice',20,85.0, 'A'), (2,'Bob', 21, 58.0,
    'C'),(3,'Charlie',22,70.0, 'B'),(4,'David',20,95.0, 'A'),(5,'Eve',21,40.0,
    'F')],dtype=student_dtype) print("Student
Database:\n", students)
```

```
#Extract only the numeric columns numeric_students =
students[['ID', 'Age', 'Marks']] print("Numeric Student Database:\n",
numeric_students)
```

3.d

```
# Get row numbers where Marks are greater than 60 rows_with_high_marks
= np.where(numeric_students['Marks']>60)[0]
print("Row numbers with Marks > 60:", rows_with_high_marks)
import pandas as pd
import numpy as np

# ------------------------------------------------
# Part A: Loading CSV File as a Pandas DataFrame
# ------------------------------------------------
```

```
# Create Traffic Flow dataset
traffic_data = {
    'Date': [
        '01-01-2024', '01-01-2024', '01-01-2024',
        '01-01-2024', '01-01-2024', '01-01-2024',
        '01-01-2024', '01-01-2024', '01-01-2024',
        '01-01-2024'
    ],
    'Time': [
        '08:00', '09:00', '10:00', '11:00', '12:00',
        '13:00', '14:00', '15:00', '16:00', '17:00'
    ],
    'Vehicle Type': [
        'Car', 'Bus', 'Car', 'Truck', 'Bike',
        'Bus', 'Car', 'Truck', 'Bike', 'Car'
    ],
    'Traffic Flow': [
        125, 85, 142, 67, 156, 98, 110, 134, 76, 145
    ]
}

# Create DataFrame and save as CSV
df = pd.DataFrame(traffic_data)
df.to_csv('traffic_flow.csv', index=False)

# Load CSV file
df = pd.read_csv('traffic_flow.csv')
```

```
# Display basic information
print("Basic Information about the DataFrame:")
df.info()

# Display first five rows
print("\nFirst 5 rows of the DataFrame:")
print(df.head())

# Display statistical summary
print("\nStatistical Summary of the DataFrame:")
print(df.describe())

arr = np.array([
    125, 85, 142, 67, 156, 98,
    110, 134, 76, 145, 120
])

max_element = np.max(arr)
min_element = np.min(arr)

print("\nMaximum Traffic Flow:", max_element)
print("Minimum Traffic Flow:", min_element)

# ------------------------------------------------
# Part C: Adding Rows and Columns in NumPy Array
# ------------------------------------------------

# Reshape array into a 2D array
```

```
arr_2d = arr.reshape(-1, 1)


print("\nOriginal NumPy Array:")
print(arr_2d)


# Add new rows
new_row = np.array([
    [100],
    [115],
    [130]
])


arr_with_new_row = np.vstack([
    arr_2d,
    new_row
])


print("\nNumPy Array after Adding New Rows:")
print(arr_with_new_row)


# Add a new column
new_column = np.arange(14).reshape(-1, 1)


arr_with_new_column = np.hstack([
    arr_with_new_row,
    new_column
])
```

```
print("\nNumPy Array after Adding New Column:")
print(arr_with_new_column)


# ------------------------------------------------
# Part D: Adding, Removing and Updating Using Pandas
# ------------------------------------------------

# Add a new row
new_row_df = pd.DataFrame({
    'Date': ['02-01-2024'],
    'Time': ['18:00'],
    'Vehicle Type': ['Bus'],
    'Traffic Flow': [100]
})


df = pd.concat([df, new_row_df], ignore_index=True)

print("\nDataFrame after Adding a New Row:")
print(df.tail())

# Remove a column
df = df.drop(columns=['Vehicle Type'])
print("\nDataFrame after Removing Vehicle Type Column:")
print(df.head())
# Add a new column
df['New_Column'] = np.arange(len(df))
print("\nDataFrame after Adding a New Column:")
print(df.head())
```

```
# Update a specific cell
df.loc[0, 'Traffic Flow'] = 999

print("\nDataFrame after Updating a Specific Cell:")
print(df.head())

import numpy as np
import matplotlib.pyplot as plt

# Part A: Creating Electricity Load Data

electricity_data = {
    'Date': [
        '2026-01-01', '2026-01-01', '2026-01-01',
        '2026-01-01', '2026-01-01'
    ],
    'Time': [
        '00:00:00', '01:00:00', '02:00:00',
        '03:00:00', '04:00:00'
    ],
    'Electricity Load': [120, 110, 105, 100, 115],
    'Temperature': [25, 24, 23, 22, 24]
}

electricity_df = pd.DataFrame(electricity_data)

electricity_df.to_csv('electricity_load.csv', index=False)
```

```
print("Electricity Load DataFrame:")
print(electricity_df)

# Part B: Correlation for Dataset 1

print("\nElectricity Load Dataset 1:")
print(electricity_df.head())

numeric_df = electricity_df[
    ['Electricity Load', 'Temperature']
]

corr1 = numeric_df.corr()

print("\nCorrelation Matrix for Dataset 1:")
print(corr1)

# Part C: Second Electricity Load Dataset

electricity_data2 = {
    'Hour': [0, 1, 2, 3, 4, 5, 6, 7],
    'Temperature': [24, 23, 22, 22, 23, 25, 28, 30],
    'Humidity': [70, 72, 75, 78, 76, 72, 68, 65],
    'Electricity Load': [100, 95, 90, 92, 105, 125, 150, 175]
```

```
electricity_df2 = pd.DataFrame(electricity_data2)

electricity_df2.to_csv(
    'electricity_load_dataset2.csv', index=False
)

electricity_df2 = pd.read_csv(
    'electricity_load_dataset2.csv'
)

print("\nElectricity Load Dataset 2:")
print(electricity_df2.head())

# Calculate Correlation Matrix
numeric_df2 = electricity_df2[
    ['Hour', 'Temperature', 'Humidity', 'Electricity Load']
]

corr2 = numeric_df2.corr()

print("\nCorrelation Matrix for Dataset 2:")
print(corr2)

# Part D: Generating Histograms
```

```
electricity_df[
    ['Electricity Load', 'Temperature']
].hist(figsize=(8, 5))

plt.suptitle('Histograms of Electricity Load Dataset 1')
plt.tight_layout()
plt.show()

electricity_df2.hist(figsize=(10, 8))

plt.suptitle('Histograms of Electricity Load Dataset 2')
plt.tight_layout()
plt.show()

house_data = {
    'Area': [1000, 1200, 1500, 1800, 2000,
             2200, 2500, 2800],
    'Bedrooms': [2, 2, 3, 3, 3, 4, 4, 5],
    'Bathrooms': [1, 2, 2, 2, 3, 3, 3, 4],
    'Price': [200000, 250000, 300000, 350000,
              400000, 450000, 500000, 600000]
}

house_df = pd.DataFrame(house_data)

# Save the data as a CSV file
house_df.to_csv('house_price.csv', index=False)
```

```
  # Read the CSV file
  house_df = pd.read_csv('house_price.c

print("House Price DataFrame:")
print(house_df)

  # ------------------------------------------------
  # Part B: House Price Dataset and Correlation Analysis
  # ------------------------------------------------

  print("\nHouse Price Dataset:")
  print(house_df.head())

  # Calculate correlation matrix
  house_corr = house_df.corr(numeric_only=True)

  print("\nCorrelation Matrix for House Price Dataset:")
  print(house_corr)

  # ------------------------------------------------
  # Part C: Wine Quality Dataset and Correlation Analysis
  # ------------------------------------------------

  wine_data = {
     'Fixed Acidity': [7.4, 7.8, 7.8, 11.2,
                       7.4, 7.9, 7.3, 7.5],
      'Volatile Acidity': [0.70, 0.88, 0.76, 0.28,
                           0.66, 0.60, 0.65, 0.50]
```

```
    'Citric Acid': [0.00, 0.00, 0.04, 0.56,
                    0.00, 0.06, 0.00, 0.36],
    'Residual Sugar': [1.9, 2.6, 2.3, 1.9,
                       1.8, 1.6, 1.2, 6.1],
    'Alcohol': [9.4, 9.8, 9.8, 9.8,
                9.4, 10.1, 9.5, 10.5],
    'Quality': [5, 5, 5, 6, 5, 5, 5, 6]
}

wine_df = pd.DataFrame(wine_data)

# Save the data as a CSV file
wine_df.to_csv('wine_quality.csv', index=False)

# Read the CSV file
wine_df = pd.read_csv('wine_quality.csv')

print("\nWine Quality Dataset:")
print(wine_df.head())

# Calculate correlation matrix
wine_corr = wine_df.corr(numeric_only=True)

print("\nCorrelation Matrix for Wine Quality Dataset:")
print(wine_corr)
```

```
# ------------------------------------------------

# Part D: Generate Histograms

# ------------------------------------------------


# Histograms for House Price Dataset

house_df.hist(figsize=(10, 8))


plt.suptitle('Histograms of House Price Dataset')

plt.tight_layout()

plt.show()


# Histograms for Wine Quality Dataset

wine_df.hist(figsize=(10, 8))


plt.suptitle('Histograms of Wine Quality Dataset')

plt.tight_layout()

plt.show(
```